

README.txt

Real-Time Flight Delay Prediction and Alerting System

MCSP-232 | MCA (MCA_NEW) | IGNOU

Student : Arpit

Enrolment: 2450998927

Guide : Rohit Yadav

Year : 2025

PROJECT OVERVIEW

This is a full-stack web application that predicts flight delays in real time using Machine Learning and a custom Propagating Delay Analyzer. Users can subscribe to flights and receive email alerts when delay risk crosses a threshold.

Key Features:

- User registration and login (Token Authentication)
- ML-based delay prediction (XGBoost model)
- Propagating Delay Analyzer (AeroDataBox API)
- Real-time weather integration (OpenWeatherMap API)
- Email alerting service (APScheduler background task)
- React.js front-end (Login, Search, Dashboard)

TECH STACK

Back-End : Python 3.14, Django 5.x, Django REST Framework

Front-End : React.js 18.x

Database : SQLite (development) / MySQL (production)

ML : XGBoost, Scikit-learn, Pandas, NumPy

Scheduler : APScheduler

APIs : AeroDataBox, OpenWeatherMap

PROJECT STRUCTURE

```
flight_predictor/
|-- backend/ Django project settings
|-- users/ Module 1: User auth (register/login)
|-- prediction/ Module 3+4: ML engine + propagating delay
|   |-- predictor.py FlightDelayPredictor class
|   |-- views.py /api/predict/ endpoint
|   |-- model.pkl Trained XGBoost model
|   |-- api_clients.py AeroDataBox + OpenWeatherMap clients
|-- flights/ Flight registry model
|-- subscriptions/ Module 5: Subscription management
|-- alerts/ Module 6: Alerting service + scheduler
|-- manage.py Django CLI entry point
|-- db.sqlite3 SQLite development database
```

```
|-- .env API keys (DO NOT commit to Git)
|-- requirements.txt Python dependencies
`-- test_api.http VS Code REST Client test file
```

frontend/ React.js application

```
|-- src/
| |-- components/ LoginPage, SearchPage, Dashboard
| `-- App.jsx Router and token state management
`-- package.json
```

PREREQUISITES

- Python 3.8 or higher
- Node.js 18.x or higher
- pip (Python package manager)
- Git

SETUP AND INSTALLATION

STEP 1 – Clone the repository

```
git clone
cd flight_predictor
```

STEP 2 – Create and activate virtual environment

Windows:

```
python -m venv venv
venv\Scripts\activate
```

Linux / macOS:

```
python -m venv venv
source venv/bin/activate
```

STEP 3 – Install Python dependencies

```
pip install -r requirements.txt
```

STEP 4 – Create the .env file

Create a file named .env in the root directory and add:

```
AERODATABOX_API_KEY=your_aerodatabox_key_here
OPENWEATHERMAP_API_KEY=your_openweathermap_key_here
```

Note: Never commit the .env file to version control.
The .gitignore file should include .env

STEP 5 – Apply database migrations

```
python manage.py migrate
```

This applies all 18 migrations and creates the SQLite database (db.sqlite3) with all required tables.

STEP 6 – Run the Django development server

python manage.py runserver

The server will start at: http://127.0.0.1:8000/

STEP 7 – Set up and run the React front-end

Open a new terminal window:

```
cd frontend
npm install
npm start
```

The React app will start at: http://localhost:3000/

----- API ENDPOINTS

All endpoints run on: http://127.0.0.1:8000

1. Register a new user

POST /api/users/register/

Body: { "email": "user@example.com", "password": "yourpass" }

2. Login

POST /api/users/login/

Body: { "email": "user@example.com", "password": "yourpass" }

Returns: { "token": "...", "email": "...", "message": "..." }

3. Predict flight delay (requires auth token)

POST /api/predict/

Headers: Authorization: Token

Body:

```
{
  "flight_number": "UA 137",
  "Airline": "UA",
  "AirportFrom": "JFK",
  "AirportTo": "LAX",
  "DayOfWeek": 5,
  "Time": 700,
  "Length": 300
}
```

Returns:

```
{
  "flight_number": "UA 137",
  "base_probability": 0.146,
  "final_delay_risk": 0.1752,
  "propagating_status": "Delayed"
}
```

You can test all endpoints using the included test_api.http

file with the VS Code REST Client extension.

BACKGROUND SERVICES

The alerting and ingestion services start automatically when the Django server runs. You will see this in the terminal:

```
Running scheduled check for flight alerts...
Alert check complete.
```

- Alert check : runs every 5 minutes
- Data ingestion: runs every 15 minutes

SWITCHING TO MYSQL (PRODUCTION)

1. Create a MySQL database and user:

```
CREATE DATABASE flight_delay_db CHARACTER SET utf8mb4;
CREATE USER 'flight_app'@'localhost' IDENTIFIED BY 'pass';
GRANT SELECT,INSERT,UPDATE,DELETE ON flight_delay_db.*
TO 'flight_app'@'localhost';
```

2. Update backend/settings.py DATABASES section:

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',
        'NAME': 'flight_delay_db',
        'USER': 'flight_app',
        'PASSWORD': 'pass',
        'HOST': 'localhost',
        'PORT': '3306',
    }
}
```

3. Run migrations again:

```
python manage.py migrate
```

TESTING THE API

Open test_api.http in VS Code with the REST Client extension.

Run the requests in order:

1. Register user
2. Login (copy the token from the response)
3. Predict (paste the token in the Authorization header)

KNOWN ISSUES

- AeroDataBox free tier has rate limits. If you exceed them, the propagating delay check will return 'Unknown' and the system will fall back to the base ML probability only.
- The ML model (model.pkl) must be present in the

prediction/ directory before starting the server.
If missing, the predict endpoint will return HTTP 500.

CONTACT

Student : Arpit
Email : hg092097@gmail.com
Guide : Rohit Yadav
Course : MCSP-232, MCA (MCA_NEW), IGNOU
